RACER-GT Mathematical Appendix

Derivations, Proofs, and Estimator Details

Yi-Hao Lai

Department of Finance, Da-Yeh University

2026-07-27 • version 1.2.0

Abstract

This appendix collects the derivations that the methodology manuscript states without full working, and documents the exact estimator each software routine implements. It is written to be checkable: every result is tied to the module and function that computes it, so a reader can verify that the code and the mathematics agree. Where the implementation departs from the textbook estimator—and it does so in three places—the departure is stated and justified rather than left for the reader to discover.

Contents

1 Notation

Table 1: Symbols used throughout.

Symbol	Domain	Meaning
t	$1, \dots, T$	historical calendar date
i	$1, \dots, N$	complete pull (one full history reconstruction)
j, k	$1, \dots, J$	chunk (a fixed query window)
d, s, r	—	collection day, stream, technical replicate
Y_{jt}	$[0, 100]$	raw GT value, date t, chunk j
$a_j,\ \ell_j$	$\mathbb{R}_{>0}, \mathbb{R}$	chunk scale and its logarithm, $\ell_j = \log a_j$
X_t	$\mathbb{R}_{>0}$	latent common-scale signal
Z_{it}	$\mathbb{R}$	calibrated value of pull i at date t
$\boldsymbol{\Sigma}$	$\mathbb{R}^{N\times N}$	residual covariance across pulls
$\boldsymbol{w}$	Δ^{N-1}	consensus weights, $\mathbf{1}^\top \boldsymbol{w} = 1$
$c_{\min}$	$\mathbb{R}_{>0}$	minimum raw value admitted to an overlap edge
$m_{\min}$	$\mathbb{N}$	minimum usable overlap days for an edge

2 Robust edge estimation

2.1 The estimator

For an edge (j, k), let $r_1, \dots, r_n$ be the log ratios $\log Y_{jt} - \log Y_{kt}$ over the usable overlap days. The Huber location estimate solves

$$\sum_{i=1}^{n} \psi_c\left(\frac{r_i - \hat{\mu}}{\hat{\sigma}}\right) = 0, \qquad \psi_c(u) = \mathsf{max}\{-c, \mathsf{min}(c, u)\}, \tag{1}$$

by iteratively reweighted least squares. Writing $w_i = \psi_c(u_i)/u_i$ for $u_i = (r_i - \mu)/\hat{\sigma}$, the update is

$$\mu^{(m+1)} = \frac{\sum_i w_i^{(m)} r_i}{\sum_i w_i^{(m)}}, \tag{2}$$

which is a weighted mean that downweights observations beyond $c\hat{\sigma}$. The scale $\hat{\sigma}$ is the normalized median absolute deviation.

2.2 Variance of the edge estimate

The asymptotic variance of an M-estimator of location is

$$\mathrm{Var}(\hat{\mu}) \ \approx\ \frac{\hat{\sigma}^2}{n} \cdot \frac{\mathbb{E}[\psi_c(u)^2]}{\big(\mathbb{E}[\psi_c'(u)]\big)^2}. \tag{3}$$

The implementation uses the finite-sample weighted counterpart: with the final Huber weights $w_i = \psi_c(u_i)/u_i$ it forms an effective sample size and divides the robust scale by it,

$$n_{\text{eff}} = \frac{\left(\sum_i w_i\right)^2}{\sum_i w_i^2}, \qquad \hat{v}_{jk} = \frac{\hat{\sigma}^2_{\text{robust}}}{\max(n_{\text{eff}}, 1)}. \tag{4}$$

The two agree closely under normality at $c = 1.345$, where the efficiency factor is about 1.04, but they are not the same estimator: (3) is asymptotic, whereas the weighted form responds directly to a handful of observations dominating a single edge. `huber_location` in `overlap.py` computes the latter. This variance becomes the edge precision $1/\hat{v}_{jk}$ in the graph problem. Two implementation details matter:

1. a floor is applied, $\hat{v} \leftarrow \max(\hat{v}, v_{\text{floor}})$, because an edge with near-zero estimated variance would otherwise dominate the normal equations entirely;
2. a ceiling is applied to the resulting weight, $1/\hat{v} \le w_{\max}$, for the same reason from the other direction.

Remark 2.1 (Departure 1 from the textbook estimator). *The floor and ceiling are not part of the classical M-estimation theory. They are numerical guards. Their effect is to bound the influence any single edge can exert on the global solution, which matters because an edge computed from exactly $m_{\min}$ nearly identical days can produce a spuriously small variance.*

3 Graph weighted least squares

3.1 Normal equations

With B the reduced incidence matrix (reference column deleted), $\boldsymbol{r}$ the stacked edge estimates and $\Omega = \text{diag}(\hat{v}_e)$,

$$(B^\top \Omega^{-1} B)\, \hat{\boldsymbol{\ell}} = B^\top \Omega^{-1} \boldsymbol{r}. \tag{5}$$

The matrix $L = B^\top \Omega^{-1} B$ is the weighted graph Laplacian with the reference row and column removed, sometimes called the grounded Laplacian.

Proposition 3.1 (Positive definiteness). *If G is connected then the grounded Laplacian L is positive definite.*

Proof. For any $\boldsymbol{x} \neq \mathbf{0}$ on the free nodes, extend it to $\tilde{\boldsymbol{x}}$ by setting the reference

entry to zero. Then $\boldsymbol{x}^\top L\boldsymbol{x} = \sum_{e=(j,k)} \hat{v}_e^{-1}(\tilde{x}_j - \tilde{x}_k)^2 \geq 0$, with equality only if $\tilde{\boldsymbol{x}}$ is constant on every connected component. Connectivity forces one component, and the reference entry pins that constant to zero, so $\tilde{\boldsymbol{x}} = \mathbf{0}$, contradicting $\boldsymbol{x} \neq \mathbf{0}$. □

3.2 Diagnostics that follow from the residual

The fitted residual $\boldsymbol{\varepsilon} = \boldsymbol{r} - B\hat{\boldsymbol{\ell}}$ is exactly the cycle inconsistency of the edge measurements: for any cycle in G, the sum of edge estimates around it should be zero, and $\boldsymbol{\varepsilon}$ measures the failure. The implementation reports:

- connectivity and the number of components;
- the condition number of L, which flags a graph held together by a single weak edge;
- the weighted residual sum of squares as a cycle-consistency statistic;
- per-node standard errors from $\mathrm{diag}(L^{-1})$.

None of these quantities exists under sequential stitching, because sequential stitching uses a spanning path and therefore has no cycles to be inconsistent about.

3.3 Lognormal bias correction

What is corrected is the Jensen bias *of the estimator*, not the median-versus-mean gap of the multiplicative error in the data-generating process. With $\hat{\ell}_j$ approximately normal and $s_j^2 = \mathrm{Var}(\hat{\ell}_j)$,

$$\mathbb{E}\Big[\exp(-\hat{\ell}_j)\Big] = \exp(-\ell_j)\exp\big(\tfrac{1}{2}s_j^2\big)\,, \tag{6}$$

so back-transforming with $\exp(-\hat{\ell}_j)$ alone systematically overstates a_j^{-1}. The first-order correction is

$$\widehat{a_j^{-1}}_{bc} = \exp\Big(-\hat{\ell}_j - \tfrac{1}{2}s_j^2\Big)\,, \tag{7}$$

as in the manuscript. The implementation takes s_j^2 from the diagonal of L^{-1} in the graph WLS solution, which is `log_scale_variance` in `overlap.py`.

Remark 3.1 (Departure 2). *This correction is exact only when $\hat{\ell}_j$ is approximately normal, and its magnitude shrinks toward zero as the number of usable overlap days grows. That distinguishes it from the variance of the observation error ϵ_{jt} itself, which does not shrink with sample size; the two are easy to conflate, so the quantity*

actually used is named explicitly here. It is enabled by default because the multiplicative model motivates it, and it remains a configuration flag because normality is not testable at these sample sizes.

4 Variance components

4.1 Expected mean squares

For the fully crossed design of the manuscript with n_t, n_d, n_s levels and n_r replicates, the method-of-moments equations follow from the expected mean squares. Writing $\sigma^2_\bullet$ for the component of the indicated effect,

$$\mathbb{E}[\mathrm{MS}_T] = \sigma^2_E + n_r\sigma^2_{TDS} + n_r n_s \sigma^2_{TD} + n_r n_d \sigma^2_{TS} + n_r n_d n_s \sigma^2_T, \tag{8}$$

$$\mathbb{E}[\mathrm{MS}_{TD}] = \sigma^2_E + n_r\sigma^2_{TDS} + n_r n_s \sigma^2_{TD}, \tag{9}$$

$$\mathbb{E}[\mathrm{MS}_{TDS}] = \sigma^2_E + n_r\sigma^2_{TDS}, \tag{10}$$

$$\mathbb{E}[\mathrm{MS}_E] = \sigma^2_E, \tag{11}$$

with the analogous equations for D, S, TS and DS. Solving downward from the highest-order term gives each component.

Remark 4.1 (Departure 3: negative components). *Method-of-moments estimates can be negative, which is impossible for a variance. The implementation clips them at zero by default. Clipping is the standard practical choice but it introduces a small upward bias in the clipped component and a corresponding distortion in any ratio that uses it. The raw, unclipped values are retained in the output so that the reader can see how often clipping was necessary; frequent clipping is evidence that the design is too small for the decomposition being attempted.*

4.2 Generalizability and dependability coefficients

For relative decisions (rank ordering across dates),

$$E\rho^2 = \frac{\sigma^2_T}{\sigma^2_T + \dfrac{\sigma^2_{TD}}{n_d} + \dfrac{\sigma^2_{TS}}{n_s} + \dfrac{\sigma^2_{TDS}}{n_d n_s} + \dfrac{\sigma^2_E}{n_d n_s n_r}}. \tag{12}$$

For absolute decisions the dependability coefficient Φ additionally charges the main effects of D and S, because a uniform shift matters when the score is interpreted

on its own rather than relative to other dates:

$$\Phi = \frac{\sigma_T^2}{\sigma_T^2 + \frac{\sigma_D^2}{n_d} + \frac{\sigma_S^2}{n_s} + \frac{\sigma_{TD}^2}{n_d} + \frac{\sigma_{TS}^2}{n_s} + \frac{\sigma_{DS}^2}{n_d n_s} + \frac{\sigma_{TDS}^2}{n_d n_s} + \frac{\sigma_E^2}{n_d n_s n_r}}. \tag{13}$$

Remark 4.2 (Why σ_{TDS}^2 and σ_E^2 carry different divisors). *The TDS interaction is averaged only over day $\times$ stream cells, so it is divided by $n_d n_s$; pure replication error is averaged n_r further times within each cell, so it is divided by $n_d n_s n_r$. Collapsing the two into a single $(\sigma_{TDS}^2 + \sigma_E^2)/(n_d n_s n_r)$ term is correct only at $n_r = 1$; for $n_r > 1$ it understates relative error variance and therefore overstates G. The confirmatory design recommended in the manuscript uses $n_r = 2$, which is exactly the case where the difference bites. equation* (12) *matches* `generalizability_coefficients` *in* `gstudy.py` *term by term, and a consistency test in* `tests/test_gstudy.py` *keeps the two from drifting apart.*

The D-study evaluates both on a grid of (n_d, n_s, n_r), which is what turns the decomposition into an actionable design recommendation: the partial derivative with respect to n_d versus n_s says where the next unit of effort buys more reliability.

5 Consensus weights under constraints

The unconstrained solution $\boldsymbol{w}^* = \boldsymbol{\Sigma}^{-1}\mathbf{1}/(\mathbf{1}^\top\boldsymbol{\Sigma}^{-1}\mathbf{1})$ is derived in the manuscript. The default configuration instead solves

$$\min_{\boldsymbol{w}} \ \boldsymbol{w}^\top \hat{\boldsymbol{\Sigma}} \boldsymbol{w} \quad \text{s.t.} \quad \mathbf{1}^\top \boldsymbol{w} = 1, \quad 0 \le w_i \le \bar{w}. \tag{14}$$

Proposition 5.1 (The constrained problem is well posed). *If $\hat{\boldsymbol{\Sigma}} \succeq 0$ and $\bar{w} \ge 1/N$, the feasible set of* (14) *is non-empty, compact and convex, so a minimizer exists. If $\hat{\boldsymbol{\Sigma}} \succ 0$ it is unique.*

Proof. The equal-weight vector $w_i = 1/N$ is feasible when $\bar{w} \ge 1/N$, so the set is non-empty; it is the intersection of a hyperplane with a box, hence compact and convex. A continuous function on a non-empty compact set attains its minimum. Strict convexity of the objective under $\hat{\boldsymbol{\Sigma}} \succ 0$ gives uniqueness. □

Remark 5.1 (Why $\bar{w}$ exists at all). *Without a cap, an estimated covariance that happens to make one pull look almost noiseless will concentrate nearly all weight on it, and the consensus degenerates toward a single pull—the very failure mode the*

framework exists to avoid. The cap trades a small amount of theoretical efficiency for the guarantee that the consensus always aggregates.

5.1 Shrinkage covariance

The Ledoit–Wolf estimator is $\hat{\Sigma} = (1-\alpha)S + \alpha F$, with S the sample covariance, F a well-conditioned target, and $\alpha \in [0,1]$ chosen to minimize expected squared Frobenius loss. It is used because N is typically small relative to the number of dates and S^{-1} is correspondingly unstable.

5.2 Effective number of pulls

Two summaries are reported:

$$N_{\text{Kish}} = \frac{1}{\sum_i w_i^2}, \qquad N_{\text{spec}} = \frac{\left(\sum_m \lambda_m\right)^2}{\sum_m \lambda_m^2}, \tag{15}$$

where λ_m are the eigenvalues of the residual correlation matrix. The first measures weight concentration; the second measures how much of the variation is shared. Both equal N only when weights are equal and residuals are uncorrelated, and both fall well below N in the presence of near-duplicates. Reporting N itself as the sample size would overstate precision, which is why the acceptance rule is stated in terms of N_{spec}.

6 Temporal benchmarking

6.1 Solution

With $Q \succ 0$, $W \succeq 0$, and A the day-to-period aggregation matrix,

$$\boldsymbol{x}^* = (Q + A^\top W A)^{-1}(Q\boldsymbol{z} + A^\top W \boldsymbol{b}). \tag{16}$$

The matrix Q is built as $(\lambda_I + \lambda_R)I + \lambda_D D_1^\top D_1$ where D_1 is the *first*-difference operator: `_difference_penalty` in `benchmark.py` forms the $(n-1)\times n$ difference matrix with offsets $[0,1]$ and returns $D_1^\top D_1$. The first term penalizes departure from the daily consensus, with λ_R the ridge that keeps Q positive definite when $\lambda_I = 0$; the second penalizes roughness in the correction path. Movement preservation is

thus imposed on the correction rather than on the level. A first-difference penalty charges the *slope* of the correction path; a second-difference penalty would charge its curvature, which is a different family of benchmarking estimators and is not what this implementation solves.

6.2 Exact mode

In exact mode the aggregate constraint $A\boldsymbol{x} = \boldsymbol{b}$ is imposed rather than penalized, giving the KKT system

$$\begin{bmatrix} Q & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{\lambda} \end{bmatrix} = \begin{bmatrix} Q\boldsymbol{z} \\ \boldsymbol{b} \end{bmatrix}, \tag{17}$$

which is solvable when A has full row rank. Soft mode is the default because the lower-frequency GT series is itself measured with error, and imposing it exactly would transfer that error into the daily series without acknowledgement.

7 Measurement-error correction

7.1 Moment correction

Let $W_t = X_t + u_t$, let M be the residual maker for the controls, and let Ω_u be the measurement-error covariance. From $\mathbb{E}[\boldsymbol{W}^\top M \boldsymbol{W}] = \mathbb{E}[\boldsymbol{X}^\top M \boldsymbol{X}] + \mathrm{tr}(M\Omega_u)$ and $\mathbb{E}[\boldsymbol{W}^\top M \boldsymbol{Y}] = \beta\, \mathbb{E}[\boldsymbol{X}^\top M \boldsymbol{X}]$,

$$\hat{\beta} = \frac{\boldsymbol{W}^\top M \boldsymbol{Y}}{\boldsymbol{W}^\top M \boldsymbol{W} - \mathrm{tr}(M\Omega_u)}. \tag{18}$$

Remark 7.1 (The denominator is a diagnostic). *A non-positive denominator means the estimated measurement error accounts for all of the observed regressor variation after partialling out the controls. The implementation raises an error instead of returning a number. Returning a sign-flipped or explosive coefficient in that situation would be worse than failing, because the failure is informative: it says the constructed index has no usable signal left for this regression.*

7.2 SIMEX

For a grid $\lambda \in \{0, \lambda_1, \ldots, \lambda_K\}$, generate $W_t(\lambda, b) = W_t + \sqrt{\lambda}\, \tilde{u}_{tb}$ with $\tilde{u}_{tb}$ drawn to match Ω_u, estimate $\hat{\beta}(\lambda, b)$ for $b = 1, \ldots, B$, and average over b. Fit a parametric curve $g(\lambda; \boldsymbol{\theta})$

to $\{(\lambda, \bar{\beta}(\lambda))\}$ and report $g(-1; \hat{\boldsymbol{\theta}})$, which extrapolates to the hypothetical case of no measurement error. SIMEX is reported alongside the moment correction rather than instead of it: agreement between the two is evidence that the classical error structure is adequate, and disagreement is evidence that it is not.

8 Correspondence between mathematics and code

Table 2: Where each result is implemented.

Result	Module	Entry point
Huber edge estimate	`overlap.py`	`huber_location`
Graph WLS, diagnostics	`overlap.py`	`OverlapGraphCalibrator.fit`
Exact/near duplicates	`duplicates.py`	`diagnose_duplicates`
Variance components, D-study	`gstudy.py`	`run_gstudy`
Design-based estimator	`consensus.py`	`fit_design_weighted_consensus`
Constrained GLS consensus	`consensus.py`	`fit_gls_consensus`
Benchmark solution	`benchmark.py`	`temporal_benchmark`
Reliability, convergence	`reliability.py`	`assess_reliability`
Acceptance decision tree	`decision.py`	`evaluate_batch`
Moment correction, SIMEX	`eiv.py`	`reliability_corrected_ols`, `simex_ols`